

MASTER_GUIDE.md

노무사 SaaS Internal HR Portal

Production Engineering Reference | v1.0 | 2025

Document Purpose

Primary architectural reference for the labor consulting firm HR/client portal.

Covers system design, implementation roadmap, security patterns, and engineering standards.

Target audience: solo developer using AI-assisted tooling (Cursor + Claude).

Sections: Architecture · Database · Security · Frontend · Backend · Excel Export · Roadmap · Testing

Table of Contents

1. Project Overview & Goals
2. High-Level System Architecture
3. Multi-Tenant Architecture & RBAC
4. Database Design
5. Security Design
6. Frontend Structure
7. Backend Structure
8. Excel Export Architecture
9. Suggested Folder Structure
10. Development Philosophy
11. Implementation Roadmap (8–12 Weeks)
12. Testing Strategy
13. Future Expansion

1. Project Overview & Goals

This application is a secure, multi-tenant HR/client portal for a Korean labor law consulting firm (노무사 사무소). It replaces manual spreadsheet workflows with a structured web application backed by a PostgreSQL database.

Business Purpose

- Client companies submit structured HR/employment data via web forms
- Labor firm staff review, manage, and export the same data
- The firm generates standardized Excel files for regulatory and operational use

MVP Scope

Three modules for MVP. Manual entry only — no Excel upload in the initial release.

Module	Korean / Description
입사자 정보	New Hire Intake — name, RRN, hire date, dept, salary type, gross/net, contract flag, contract period
퇴사자 정보	Termination Records — name, RRN, termination date, reason
급여변경	Compensation Changes — name, RRN, before/after salary, change date

Each module supports: list/table view · create/edit/delete · filtering · field validation · audit logging · Excel export

Core User Types

Role	Permissions
Client Company Admin	Log in, manage own company's records only, export own data. Cannot access any other tenant's data.
Firm Staff / Admin	Access all companies, browse all records, edit/export all data, view audit metadata and logs.

2. High-Level System Architecture

The application uses a monolithic full-stack Next.js architecture — a deliberate choice for a solo developer building a CRUD-heavy internal tool. No microservices, no separate API server, no event bus.

Why Monolithic Full-Stack Next.js

- Single deployment unit on Vercel — zero infrastructure overhead
- Server Actions colocate data-fetching logic with UI without a separate REST layer
- Prisma ORM provides type-safe database access without boilerplate
- Shared TypeScript types across client and server eliminate interface drift
- AI-assisted (Cursor) development is most effective with a predictable, uniform codebase

Layer Separation

Layer	Responsibility
UI (React Components)	Presentation only. No business logic. Calls Server Actions or fetches from Route Handlers.
Server Actions / Route Handlers	Input validation (Zod), authorization check, delegate to service layer.
Service Layer (lib/services/)	Business logic, Prisma queries, data transformations. No HTTP concerns.
Database (Neon Postgres)	Source of truth. Enforces referential integrity. Accessed only through Prisma.
Encryption Utilities (lib/crypto/)	주민번호 encryption/decryption. Isolated, tested separately.

Request Lifecycle

14. User submits a form (React Hook Form + Zod client-side validation)
15. Server Action fires — Zod validates again server-side (never trust client)
16. Authorization check: verify session, extract company_id, confirm role
17. Service function executes Prisma query scoped to company_id
18. Audit log entry written in the same transaction
19. Response returned; UI revalidates via Next.js cache tags

Key Constraint

Business logic lives exclusively in lib/services/. Server Actions are thin controllers.

This makes unit testing straightforward and prevents logic from spreading across the codebase.

3. Multi-Tenant Architecture & RBAC

Every record in the system belongs to a company. Tenant isolation is the most critical security property of this application.

Tenant Isolation Rules

- Every data table has a `company_id` (UUID, FK to companies table) column
- Every Prisma query from client-facing Server Actions must include `where: { company_id: session.companyId }`
- The `company_id` is never sourced from user input — always from the verified session
- Firm staff bypass `company_id` scoping but still require authentication and role validation

Role Definitions

Role	Behavior
CLIENT_ADMIN	Scoped to own <code>company_id</code> . All queries filtered by it. Cannot modify firm-side data.
FIRM_STAFF	Can query any company. No <code>company_id</code> filter applied. Read/write access to all modules.
FIRM_ADMIN	Same as FIRM_STAFF plus user management and system configuration access.

Authorization Pattern

Do not scatter auth checks across components. Use a single centralized helper:

```
// lib/auth/guards.ts

export async function requireAuth(role?: Role) {

  const session = await getServerSession(authOptions);

  if (!session) redirect('/login');
```

```
if (role && session.user.role !== role) throw new ForbiddenError();  
  
return session;  
  
}
```

Every Server Action calls requireAuth() at the top. No exceptions.

Why Backend Enforcement Matters

Client-side role checks (hiding UI elements, disabling buttons) are UX enhancements only. A motivated user can bypass them with developer tools. The database never receives a query without company_id enforcement at the server layer.

4. Database Design

Core Entities

Table	Purpose
companies	Tenant root. Every data record references this.
users	All users — client admins and firm staff. Has role, company_id (null for firm staff).
new_hires	입사자 정보 records. Scoped to company_id.
terminations	퇴사자 정보 records. Scoped to company_id.
compensation_changes	급여변경 records. Scoped to company_id.
audit_logs	Immutable event log. See Audit Strategy below.

Schema Conventions

- All PKs are UUID (gen_random_uuid()). Never sequential integers for tenant-scoped records.
- created_at / updated_at on all tables — set via Prisma @default(now()) and @updatedAt
- Soft deletes: deleted_at TIMESTAMP NULL. Queries default to WHERE deleted_at IS NULL.
- 주민번호 stored as rrn_encrypted (TEXT) + rrn_iv (TEXT). Never plaintext.
- String fields use VARCHAR with explicit length limits matching Zod schemas

Key Relationships

companies 1—* users

companies 1—* new_hires

companies 1—* terminations

companies 1—* compensation_changes

users 1—* audit_logs (actor_id)

Audit Log Strategy

The audit_logs table is append-only. No updates or deletes. It records every meaningful state change.

Column	Description
id	UUID primary key
actor_id	FK to users — who performed the action
company_id	Tenant context at time of action
action	Enum: CREATE, UPDATE, DELETE, EXPORT, LOGIN, etc.
table_name	Which module was affected
record_id	FK to affected record (nullable)
payload	JSONB diff — {before, after}. Exclude RRN and sensitive fields.
created_at	Immutable timestamp

Sensitive Data in Audit Logs

Never log 주민번호 — not even encrypted form — in the audit payload.

Log field names changed (e.g., 'rrn updated') without the value.

Salary values may appear in logs but consider masking based on firm policy.

Migration Strategy

- All schema changes via Prisma Migrate (prisma migrate dev in development)
- Production migrations run via CI before deployment (prisma migrate deploy)
- Never use prisma db push in production
- Keep migration files in version control — they are part of the codebase

5. Security Design

주민번호 (RRN) Handling

주민번호 is South Korean Personally Identifiable Information (PII) subject to strict legal protection. It must never be stored in plaintext.

Encryption Approach

- Algorithm: AES-256-GCM (authenticated encryption — detects tampering)
- Key: 32-byte key loaded from environment variable (ENCRYPTION_KEY). Never in source code.
- IV: Random 12-byte IV generated per record. Stored alongside ciphertext in `rrn_iv` column.
- Storage: `rrn_encrypted` (base64 ciphertext) + `rrn_iv` (base64 IV)

```
// lib/crypto/rrn.ts - conceptual only
encrypt(plaintext: string): { encrypted: string; iv: string }
decrypt(encrypted: string, iv: string): string
```

Masking Strategy

- Display: show only last 7 digits masked — e.g., 123456-*****
- Export: firm staff exports show full decrypted RRN; client exports mask it
- Logs: never log RRN in any form
- React: mask on render; full value only populated when explicitly needed

Session & Authentication

- Auth.js (NextAuth v5) with credential provider for email/password login
- Sessions stored as encrypted JWTs (jose). No server-side session storage needed for MVP.
- Session contains: `userId`, `role`, `companyId` (null for firm staff)
- Session expiry: 8 hours for client admins, 12 hours for firm staff
- Re-authentication required after expiry — no silent refresh for MVP

CSRF Protection

- Next.js App Router Server Actions have built-in CSRF protection via Origin header checking
- For Route Handlers: validate Content-Type and Origin headers explicitly
- Auth.js CSRF tokens handle form-based auth flows

Authorization Patterns

- Route-level: middleware.ts protects /client/* and /firm/* route groups
- Action-level: requireAuth() called at the start of every Server Action
- Data-level: Prisma queries always include company_id scope for CLIENT_ADMIN role

Other Security Measures

Concern	Approach
HTTPS	Enforced by Vercel. All cookies use Secure flag.
Secrets Management	All secrets in environment variables. Never in code or logs.
Password Hashing	bcrypt (cost factor 12) via Auth.js built-in handling.
Input Validation	Zod schemas validated server-side on every mutation. No trust of client data.
Rate Limiting	Add on /api/auth/* routes. Use Vercel Edge Middleware or upstash/ratelimit.
Error Responses	Never expose stack traces or DB errors to the client. Log internally via Sentry.
Dependency Audit	Run npm audit in CI. Address high/critical vulnerabilities before deploy.

6. Frontend Structure

Route Groups

Next.js App Router route groups separate client and firm portals without affecting URL paths:

```
app/

(auth) /                # Login, logout – no layout chrome

  login/page.tsx

(client) /              # Client company portal

  layout.tsx            # Client nav, session guard

  dashboard/page.tsx

  new-hires/

  terminations/

  compensation/

(firm) /                # Firm staff portal

  layout.tsx            # Firm nav, role guard

  dashboard/page.tsx

  companies/

  [companyId]/new-hires/

  exports/
```

Dashboard Structure

- Client dashboard: summary counts of own records, recent activity, quick-add buttons
- Firm dashboard: company list with record counts, recent cross-company activity, export shortcuts

- Both dashboards are server components — data fetched at render time, no client-side loading states for initial data

Table Architecture

Use a single reusable DataTable component built on TanStack Table (included with shadcn/ui). Configure per-module with column definitions.

- Column definitions live in modules/[module]/columns.tsx
- Sorting, pagination, and filtering are URL-parameter-driven (searchParams)
- Row actions (edit, delete) are rendered inline with permission-aware visibility
- RRN column always renders masked. A 'reveal' button can decrypt client-side (requires a separate API call).

Form Architecture

- React Hook Form + Zod resolver for all forms
- Zod schema defined once in lib/validations/[module].ts — shared with server-side validation
- Form components are controlled — no uncontrolled inputs
- Server Action called on submit. Errors returned as structured objects, not thrown.
- Loading states handled via useFormStatus or React transition

Reusable UI Patterns

Component	Usage
DataTable	All list views. Configured with column definitions per module.
FormModal	Create/edit forms in a Sheet or Dialog from shadcn/ui.
ConfirmDialog	Delete confirmations. Prevents accidental data loss.
StatusBadge	Contract type, salary type, termination reason indicators.
MaskedRRN	Renders masked 주민번호. Optionally decrypts on demand.
ExportButton	Triggers server-side Excel generation. Shows download progress.

Client vs Server Components

- Default to Server Components. Fetch data, render HTML. No useState, no useEffect.
- Use Client Components only for: interactive forms, dropdowns, modals, tables with client-side actions.
- Mark with 'use client' only as needed. Keep client component trees shallow.

7. Backend Structure

Server Actions vs Route Handlers

Use Server Actions for	Use Route Handlers for
Form submissions	File downloads (Excel export)
Create/update/delete mutations	External webhooks (future)
Any mutation triggered from a form or button	Programmatic API access (future)

Validation Flow

- 20.Client: Zod schema validates on blur/submit (React Hook Form resolver)
- 21.Server Action receives FormData or typed object
- 22.Server: Zod parses again — this is the authoritative validation
- 23.If invalid: return { error: 'Validation failed', fields: zodError.flatten() }
- 24.If valid: proceed to auth check, then service call

Rule

Never skip server-side validation. Client validation is UX only.

The Zod schema in lib/validations/ is the single source of truth for shape and constraints.

Prisma Service Layer

Service functions in lib/services/ are the only place Prisma is called. They:

- Accept typed parameters (never raw request objects)
- Apply company_id scoping based on role passed in
- Write audit log entries in the same transaction as the data change
- Return typed results or throw typed errors

```
// lib/services/new-hires.ts
export async function createNewHire(
  data: NewHireInput,
  actorId: string,
  companyId: string
): Promise<NewHire>
```

Centralized Error Handling

- Define a small set of typed error classes: `NotFoundError`, `ForbiddenError`, `ValidationError`
- Server Actions catch these and return structured error responses
- Unexpected errors are logged to Sentry and return a generic message to the client
- Never expose DB error messages or stack traces

Encryption Utilities

- `lib/crypto/rn.ts` exports `encrypt()` and `decrypt()` only
- The encryption key is loaded from `process.env.ENCRYPTION_KEY` at module load
- `encrypt()` is called in the service layer before Prisma write
- `decrypt()` is called only when RRN display/export is explicitly needed
- Unit test this module exhaustively — it is critical infrastructure

8. Excel Export Architecture

Excel export is a first-class feature. The firm's operational workflow depends on it. Build it correctly from day one.

Technology

- Library: `xlsx` (SheetJS) — battle-tested, no native dependencies, runs on Vercel serverless
- Generation: server-side only. Never send raw data to the client for client-side export.
- Delivery: Route Handler returns `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet` with `Content-Disposition: attachment`

Export Mapping Architecture

Define column mappings in a centralized configuration file per module. This is the single place to update when the firm's Excel template changes.

```
// lib/exports/mappings/new-hires.ts

export const NEW_HIRE_COLUMNS = [

  { header: '이름',      key: 'name',      width: 15 },

  { header: '입사일',    key: 'hireDate',  width: 14, format: 'date' },

  { header: '부서',      key: 'department', width: 20 },

  { header: '주민번호',  key: 'rrn',      width: 20, sensitive: true },

  // ...

]
```

Export Pipeline

- 25.Route Handler authenticates request, extracts companyId + filters
- 26.Service function queries records, decrypts RRN where role permits
- 27.Export service maps records to column definitions
- 28.Applies formatting: dates as YYYY-MM-DD, numbers as currency, RRN masked if client export
- 29.Generates XLSX buffer via SheetJS
- 30.Returns buffer as streaming response with correct headers

Naming Conventions

`[company_name]_[module]_[YYYYMMDD].xlsx`

Example: 삼성물산_입사자정보_20250401.xlsx

Formatting Standards

- Date columns: YYYY-MM-DD format, not locale-specific
- Salary columns: number format with comma separators, no currency symbol in cell
- Header row: bold, background fill #1E3A5F, white text
- Data rows: alternating white/light-gray for readability
- Column widths: set per mapping definition
- Sheet name: Korean module name (e.g., '입사자정보')

Future Maintainability

When the firm requests format changes, only lib/exports/mappings/ needs to be updated. The pipeline, formatting, and delivery code remain unchanged. This is the correct abstraction boundary.

9. Suggested Folder Structure

```
|— app/
```

```
|   |— (auth)/
```

```
|   |   └─ login/
```

```
|   |— (client)/
```

```
|   |   |— layout.tsx
```

```
|   |   |— dashboard/
```

```
|   |   |— new-hires/
```

```
|   |   |— terminations/
```

```
|   |   └─ compensation/
```

```
|   |— (firm)/
```

```
|   |   |— layout.tsx
```

```
|   |   |— dashboard/
```

```
|   |   |— companies/
```

```
|   |   └─ exports/
```

```
|   └─ api/
```

```
|       |— auth/[...nextauth]/
```

```
|       └─ exports/
```

| |— new-hires/route.ts

| |— terminations/route.ts

| └ compensation/route.ts

|— components/

| |— ui/ # shadcn/ui generated components

| |— data-table/ # Reusable table + column types

| |— forms/ # Shared form primitives

| └ layout/ # Nav, sidebar, shell

|— lib/

| |— auth/

| | |— config.ts # Auth.js options

| | └ guards.ts # requireAuth()

| |— crypto/

| | └ rrn.ts # encrypt / decrypt

| |— db/

| | └ prisma.ts # Prisma singleton

| |— exports/

```
| | └─ mappings/          # Per-module column definitions
```

```
| | └─ excel.ts          # Core XLSX generation
```

```
| └─ services/          # All Prisma queries live here
```

```
| | └─ new-hires.ts
```

```
| | └─ terminations.ts
```

```
| | └─ compensation.ts
```

```
| └─ validations/        # Zod schemas (shared client+server)
```

```
| └─ new-hire.ts
```

```
| └─ termination.ts
```

```
| └─ compensation.ts
```

```
└─ modules/              # Module-specific UI (colocated)
```

```
| └─ new-hires/
```

```
| | └─ columns.tsx      # TanStack column defs
```

```
| | └─ form.tsx         # Create/edit form
```

```
| | └─ actions.ts       # Server Actions for this module
```

```
| └─ terminations/
```

```
| └─ compensation/
```

|— prisma/

| |— schema.prisma

| |— migrations/

|— middleware.ts # Route protection

|— types/

| |— index.ts # Shared TypeScript types

10. Development Philosophy

Core Principles

Principle	What it means in practice
No overengineering	Build what the MVP needs. Add abstractions only when you have three or more concrete use cases for them.
Modular monolith	Single deployable unit. Logical separation by module. No microservices until traffic/team forces it.
Convention consistency	Same file names, same function signatures, same error patterns across all modules. New modules are copy-and-adapt, not reinvent.
Strict typing	No any. Enable strict mode in tsconfig. Types flow from Prisma schema outward — don't duplicate type definitions.
Centralized business logic	lib/services/ is the authority. Components and Server Actions are thin. Tests target services.
Maintainability over cleverness	Readable, boring code is a feature. The next developer (or future you) will thank you.

AI-Assisted Development Best Practices

- Give Cursor/Claude full file context — don't paste snippets
- Reference this guide in prompts: 'following the patterns in MASTER_GUIDE.md'
- Generate one module completely, then use it as the template for others
- Always review AI output for security-sensitive code (auth, encryption, Prisma queries)
- Commit working code frequently — AI-generated code can regress quickly
- Write the test for a function, then ask AI to implement it — TDD works well with AI

What to Avoid

- Custom ORM wrappers around Prisma — Prisma is already excellent
- Redux or Zustand for server state — use React Query or Next.js cache
- Multiple UI component libraries — shadcn/ui is sufficient
- Event sourcing, CQRS, or DDD patterns — overkill for CRUD
- Docker for local development (unless team has multiple OS environments)

11. Implementation Roadmap

Timeline Estimate

8–12 weeks for a solo AI-assisted developer to reach production-ready pilot.

Testing, debugging, and security hardening typically consume 35–40% of total effort.

Do not underestimate Phase 6 (Security) and Phase 7 (QA).

Phase 1 — Planning + Architecture (Week 1)

Item	Detail
Goals	Finalize requirements, confirm tech stack, design database schema
Deliverables	ERD, finalized Zod schemas, Auth.js config plan, environment setup
Testing Focus	Schema review against business rules
Common Pitfalls	Skipping schema design — changes later are expensive. Under-specifying RRN encryption approach.

Phase 2 — Project Foundation (Week 1–2)

Item	Detail
Goals	Working Next.js project, DB connected, Auth.js login flow, route groups, middleware
Deliverables	Login page, session handling, protected routes, Prisma schema with migrations, Sentry connected
Testing Focus	Auth flow, redirect behavior, session content
Common Pitfalls	Auth.js configuration edge cases. Neon connection pooling setup for serverless. Middleware regex mistakes.

Phase 3 — CRUD Modules (Week 2–4)

Item	Detail
Goals	All three modules fully functional for client portal
Deliverables	List views, create/edit/delete, RRN encryption, validation, audit logging
Testing Focus	Validation edge cases, RRN encryption/decryption, soft delete behavior, audit trail correctness
Common Pitfalls	Copy-paste errors between modules (test each independently). Forgetting company_id scope on queries.

Phase 4 — Firm Staff Portal (Week 4–5)

Item	Detail
Goals	Firm-side views with cross-company access, company management, audit log viewer
Deliverables	Firm dashboard, company list, cross-company record views, RRN masking/reveal for firm role
Testing Focus	Role boundaries, tenant isolation (firm staff should NOT bypass data integrity), audit log display
Common Pitfalls	Accidentally showing firm-side routes to client users. Over-permissioning firm staff on delete operations.

Phase 5 — Excel Export System (Week 5–6)

Item	Detail
Goals	Functional Excel export for all three modules, with correct formatting and RRN handling
Deliverables	Export Route Handlers, column mapping configs, formatted XLSX output, file naming
Testing Focus	Output format correctness, RRN masking in client exports, large dataset performance
Common Pitfalls	SheetJS column width units vs pixels. Date formatting locale issues. Memory on large exports (stream if needed).

Phase 6 — Security Hardening (Week 6–7)

Item	Detail
------	--------

Goals	Production-grade security posture
Deliverables	Rate limiting on auth routes, security headers, dependency audit, encryption key rotation plan, penetration test checklist
Testing Focus	RBAC bypass attempts, tenant isolation stress test, session expiry behavior, error message leakage
Common Pitfalls	Treating this as optional. Missing Content-Security-Policy header. Not testing token expiry flows.

Phase 7 — QA + Stabilization (Week 7–9)

Item	Detail
Goals	Confidence in production correctness
Deliverables	Seeded test data (realistic Korean HR data), end-to-end test scripts, bug fixes, performance review
Testing Focus	Full user journeys for both roles. Export accuracy. Edge cases: empty states, 0 records, very long names, duplicate RRNs.
Common Pitfalls	Testing only the happy path. Not testing the firm portal with 50+ companies. Skipping mobile viewport checks.

Phase 8 — Pilot Deployment (Week 9–12)

Item	Detail
Goals	Live pilot with one client company on production infrastructure
Deliverables	Vercel production deployment, Neon production DB, env vars set, monitoring live, onboarding docs for pilot client
Testing Focus	Production smoke test, Sentry error monitoring, DB connection under real load
Common Pitfalls	Missing production env vars. Neon connection pool exhaustion. Sentry DSN pointing to dev project.

12. Testing Strategy

For a solo developer, testing focuses on correctness of critical paths rather than 100% coverage. Prioritize tests that prevent data breaches and data corruption.

RBAC Testing

- Test that CLIENT_ADMIN cannot access /firm/* routes
- Test that CLIENT_ADMIN Server Actions reject requests with a different company_id
- Test that FIRM_STAFF can access all companies without error
- Test role escalation: a CLIENT_ADMIN cannot promote themselves

Tenant Isolation Testing

- Create two companies, each with an admin. Confirm Company A's admin cannot read Company B's records via direct API calls
- Test with Prisma queries directly — construct requests that omit company_id and confirm they are rejected at the service layer

Encryption Testing

- Unit test encrypt() and decrypt() in lib/crypto/rmn.ts
- Test that encrypted values differ even for the same plaintext (random IV)
- Test that decryption with wrong key fails gracefully
- Test that RRN is not present in plaintext in the database after a create operation

Export Validation

- Seed a known dataset and export to Excel. Parse the Excel and assert column values match source data.
- Confirm RRN is masked in client exports and unmasked in firm exports
- Test empty export (0 records) — should produce a valid XLSX with headers only

Form Validation

- Test all Zod schemas with valid and invalid inputs — including boundary values
- Test Korean-language field inputs (Unicode, special characters in names)

- Test RRN format validation (13-digit, numeric only)

Auth/Session Testing

- Test login with correct credentials
- Test login with wrong password — confirm error message does not reveal whether user exists
- Test session expiry — expired sessions should redirect to login
- Test that protected routes are unreachable without a session

Realistic Seeded Test Data

- Seed at least 3 client companies
- Each company should have 20+ records per module
- Use realistic Korean names and valid-format RRNs (not real RRNs)
- Include edge cases: employees with identical names, null optional fields, maximum-length strings

Production QA Checklist

- All pages render without console errors
- All forms submit and show correct success/error states
- Excel exports download and open correctly in Microsoft Excel (not just Numbers/LibreOffice)
- Mobile viewport usability check (tables scroll horizontally, forms are usable)
- Sentry captures a test error in production environment

13. Future Expansion

The architecture is designed to accommodate growth without requiring rewrites. Specific extension points:

Feature	Architectural Fit
Insurance Calculators	Add a new /calculators route group. Calculator logic lives in lib/services/calculators/. No schema changes needed.
Payroll Tools	New module following the same pattern as existing modules. Add payroll table to Prisma schema. Reuse DataTable + FormModal components.
Approval Workflows	Add a status column to relevant tables (PENDING, APPROVED, REJECTED). Add approvals table for workflow history. FIRM_ADMIN role gets approve/reject actions.
Notifications	Add a notifications table. Trigger entries in service functions after key events. Add a notification bell component reading from this table.
Bulk Operations	Route Handler accepts array input. Service function wraps in Prisma transaction. Add progress indicator for large batches.
Additional HR Forms	Each new form is a new module folder under modules/ with its own schema, service, columns, and actions. The pattern is established and repeatable.
Excel Upload (Future)	Parse uploaded XLSX on the server using SheetJS. Map to Zod schema for validation. Bulk-insert via service layer. Client-side preview before commit.
API for External Integrations	Add versioned Route Handlers under /api/v1/. Use API key auth (separate from session auth). Rate-limited independently.

Scaling Note

The monolithic architecture supports up to ~50 concurrent client companies without infrastructure changes.

Beyond that: Neon connection pooling (PgBouncer), Vercel Pro for larger serverless limits, and read replicas for exports.

Microservices are not needed until the team grows beyond 3–4 developers or modules become independently deployable business units.